CAPSTONE PROJECT FINAL REPORT

WorkHub – Team Workflow & Event Operations Management System

A Full-Stack Collaborative Enterprise Platform for Mega-Events, Cross-Functional Departments, and Operational Portfolios

---

Candidate Name: Namitha Koduru

Academic Discipline: Computer Science & Engineering

Project Category: Full-Stack Web Systems (MERN Stack Architecture)

Institutional Reference: Vignan University / Engineering Faculty

Primary Case Study: Vignan Mahotsav 2027 Mega-Fest Operations

Submission Date: November 2026

Version: 1.0.0 (Production Release)

CORE ARCHITECTURAL HIGHLIGHTS

- Strict MERN Stack Implementation: MongoDB (Mongoose 8.x), Express.js, React 18 (Vite 6), Node.js (v22).
- Pure Deterministic Engineering: Traditional software without AI dependencies or black-box APIs.
- 4-Tier Operational Hierarchy: Organization → Workspace → Team/Department → Collaborative Rooms.
- 3 Collaborative Onboarding Workflows: Shareable Invite Links, Unique Member IDs (WH-XXXXXX), and Directory Search.
- Comprehensive Quality Assurance: 20/20 Passing Automated Integration Test Suites & 0 Vite Warnings.

CERTIFICATE OF ORIGINALITY

This is to certify that the capstone project entitled **"WorkHub – Team Workflow & Event Operations Management System"** submitted by **Namitha Koduru** in partial fulfillment of the academic requirements, is a bona fide record of independent and original full-stack software development carried out under rigorous engineering methodologies. All modules, data models, API endpoints, and client interfaces documented herein represent actual implemented source code without reliance on synthetic mock data or external artificial intelligence generators.

Author / Developer:

Namitha Koduru
Department of Computer Science & Engineering

_____
Signature of Candidate

Project Evaluator / Academic Guide:

Department Review Committee
School of Computing & Engineering

_____
Signature of Evaluator

TABLE OF CONTENTS

1. Abstract	**3**
2. Introduction & Background	**4**
3. Problem Statement & Industrial Need	**5**
4. Existing Systems vs. Proposed WorkHub System	**6**
5. Project Objectives & Operational Scope	**7**
6. Functional & Non-Functional Requirements	**8**
7. Technology Stack & Architectural Rationale	**9**
8. High-Level System Architecture & Component Interactions	**10**
9. Use Case Modeling & Actor Interaction Specifications	**11**
10. Entity-Relationship (ER) Modeling & Data Constraints	**12**
11. Relational Database Design & Mongoose Schemas	**13**
12. Application Module Breakdown & Feature Specifications	**15**
13. Implementation Details & Technical Innovations	**17**
14. Verification, Testing & Test Cases Execution (20/20 Passed)	**19**
15. Application Visual Walkthrough & Real Screenshots	**21**
16. Performance Results, Velocity Metrics & Analysis	**25**
17. Conclusion & Future Roadmap	**26**
18. References & Technical Citations	**27**

1. Abstract

In contemporary organizational management, coordinating multi-faceted, high-stakes initiatives such as annual collegiate festivals (e.g., *Vignan Mahotsav 2027* with over 15,000 attendees), academic symposia, and enterprise projects represents a severe operational bottleneck. Standard personal productivity tools fail to provide hierarchical synchronization, leading to communication fragmentation, redundant effort, untracked dependencies, and delayed task sign-offs. This capstone engineering project presents **WorkHub**, an enterprise-grade collaborative team workflow and operations management system architected on the modern MERN stack (MongoDB, Express.js, React 18 with Vite, and Node.js with Socket.IO).

WorkHub introduces a structured four-tier organizational hierarchy: **Organization** → **Workspace** → **Team/Department** → **Collaborative Rooms**. Each room functions as an isolated execution unit with dedicated Kanban boards, multi-assignee task distribution, milestone tracking, and audit logging. Crucially, the platform features three novel collaborative onboarding mechanisms: (1) Shareable invite links, (2) Direct addition via Unique Member IDs (WH-XXXXXX), and (3) An organizational discovery directory with manager approval gates. A five-tier Role-Based Access Control (RBAC) engine enforces least-privilege governance. Strictly engineered without AI components, WorkHub delivers deterministic accountability, master event calendar scheduling, multi-collaborator review pipelines, and consolidated daily standup blocker triaging. Automated verification confirms 100% test pass rates across 20 integration test suites, establishing WorkHub as a robust, scalable software engineering solution for collaborative operations.

2. Introduction & Background

Large-scale project planning involves complex interdependencies between physical infrastructure, financial sponsorships, creative media, technical engineering, and public relations. In educational institutions and corporate departments alike, managing events requires synchronizing hundreds of active personnel across distinct specialized committees.

Case Study: Vignan Mahotsav 2027
To provide a rigorous real-world validation baseline, WorkHub was designed and seeded around the complete end-to-end execution of Vignan Mahotsav 2027. This event encompasses seven core departments: PR & Digital Marketing, Technical Team, Hospitality, Logistics & Infrastructure, Sponsorship & Finance, Cultural Events, and Sports Tournaments. Inside each department, specialized working cells (rooms) execute granular objectives—such as Stage Sound & Lighting, Network Mesh Deployment, and Press Releases. WorkHub provides the operational spine connecting executive coordinators with student organizers.

3. Problem Statement & Industrial Need

Existing project coordination mechanisms suffer from critical structural deficiencies when applied to collaborative event operations:

- **Siloed Communication:** Reliance on messaging apps (WhatsApp, Telegram) creates untracked action items, loss of context, and missing accountability.
- **Flat vs. Hierarchical Mismatch:** Conventional task managers (Trello, Todoist) model projects as flat task lists, failing to reflect multi-departmental corporate/university hierarchies.

- **Spreadsheet Fragility:** Excel/Google Sheets lack concurrency control, status progression state machines, multi-party review approvals, and real-time socket updates.
- **Inflexible Member Enrolment:** Administrative overhead in manually creating user accounts discourages volunteer onboarding, while unrestricted open links risk security breaches.
- **Absence of Daily Standup Synthesis:** Executive coordinators cannot efficiently aggregate morning standups and blocker flags across 10+ teams into an actionable overview.

4. Existing Systems vs. Proposed WorkHub System

Feature / Dimension	Traditional Solutions (Trello, Sheets)	WorkHub Operations Platform
Structural Hierarchy	Flat board or disconnected sheets with no departmental nesting.	**4-Tier: Org → Workspace → Team → Room.**
Member Onboarding	Manual email invitation by global admins only.	**3 Methods: Invite Link, Unique ID (WH-XXXXXX), Directory Search.**
Task Lifecycle	Binary checked/unchecked or arbitrary unverified movement.	**5-Tier verified pipeline with reviewer sign-off & change requests.**
Daily Synchronization	Scattered oral morning meetings with no persisted logs.	**Dedicated Today/Tomorrow planner & workspace blocker triage table.**
Master Calendar	Personal calendar disconnected from departmental deadlines.	**Multi-scope FullCalendar aggregating meetings, activities, and tasks.**
Access Control	Coarse binary permissions (Admin or Member).	**Granular 5-Tier RBAC with departmental data isolation.**

5. Project Objectives & Operational Scope

Core Engineering Objectives:
1. **Architectural Cohesion:** Deliver a full-stack MERN application connecting React 18, Express.js REST API, and MongoDB with 100% data fidelity.
2. **Hierarchical Multi-Tenancy:** Enforce data isolation such that room leaders only manage authorized rooms, while workspace managers obtain global portfolio metrics.
3. **Collaborative Onboarding:** Implement self-service join links, direct Unique Member ID adding, and directory search with approval workflows.
4. **Approval State Machine:** Provide formal review gates preventing premature task sign-offs without leader inspection.
5. **Deterministic Execution:** Eliminate AI hallucination risks by relying entirely on deterministic business logic, validation schemas, and automated test coverage.

6. Functional & Non-Functional Requirements

Functional Requirements (FR)

- **FR-01 (Authentication & RBAC):** User registration with bcrypt hashing, JWT cookie authorization, and 5-tier role assignment.
- **FR-02 (Workspace Management):** CRUD operations on workspaces, archiving, operational progress computation, and metric aggregation.
- **FR-03 (Team & Room Provisioning):** Departmental roster management, mandate checklists, room channel isolation, and leader delegation.
- **FR-04 (Task Pipeline & Kanban):** Multi-collaborator task creation, subtask checkpoint toggling, drag-and-drop status mutation, and due-date tracking.
- **FR-05 (Review & Approval Flow):** Formal submission for review, feedback note posting, change request cycles, and verified completion.
- **FR-06 (Calendar & Scheduling):** Master event calendar, personal schedule view, meeting agendas, and deadline synchronization.
- **FR-07 (Daily Work Planner):** Today accomplishment logging, in-progress tasks, tomorrow work plans, and blocker categorization.
- **FR-08 (Real-Time Push):** Socket.IO event dispatching for room activity, task movement, announcements, and notification toasts.
- **FR-09 (Onboarding Engine):** Invite link generation, unique Member ID lookup (WH-XXXXXX), and workspace join request management.

Non-Functional Requirements (NFR)

- **NFR-01 (Performance & Latency):** REST API response times < 150ms under typical loads; client bundle load time < 2.0s via Vite code-splitting.
- **NFR-02 (Security & Data Integrity):** HttpOnly cookie storage for JWT, CORS origin whitelisting, Helmet HTTP security headers, and Zod schema sanitization.
- **NFR-03 (Availability & Failover):** Resilient database connector with automated DNS resolution and local MongoDB fallback when cloud Atlas is restricted.

- **NFR-04 (Usability & Responsiveness):** Fully responsive Tailwind CSS layout across mobile (375px), tablet (768px), and desktop (1440px) viewports.
- **NFR-05 (Maintainability):** Clean MVC controller separation, modular component hierarchy, and zero circular dependencies.

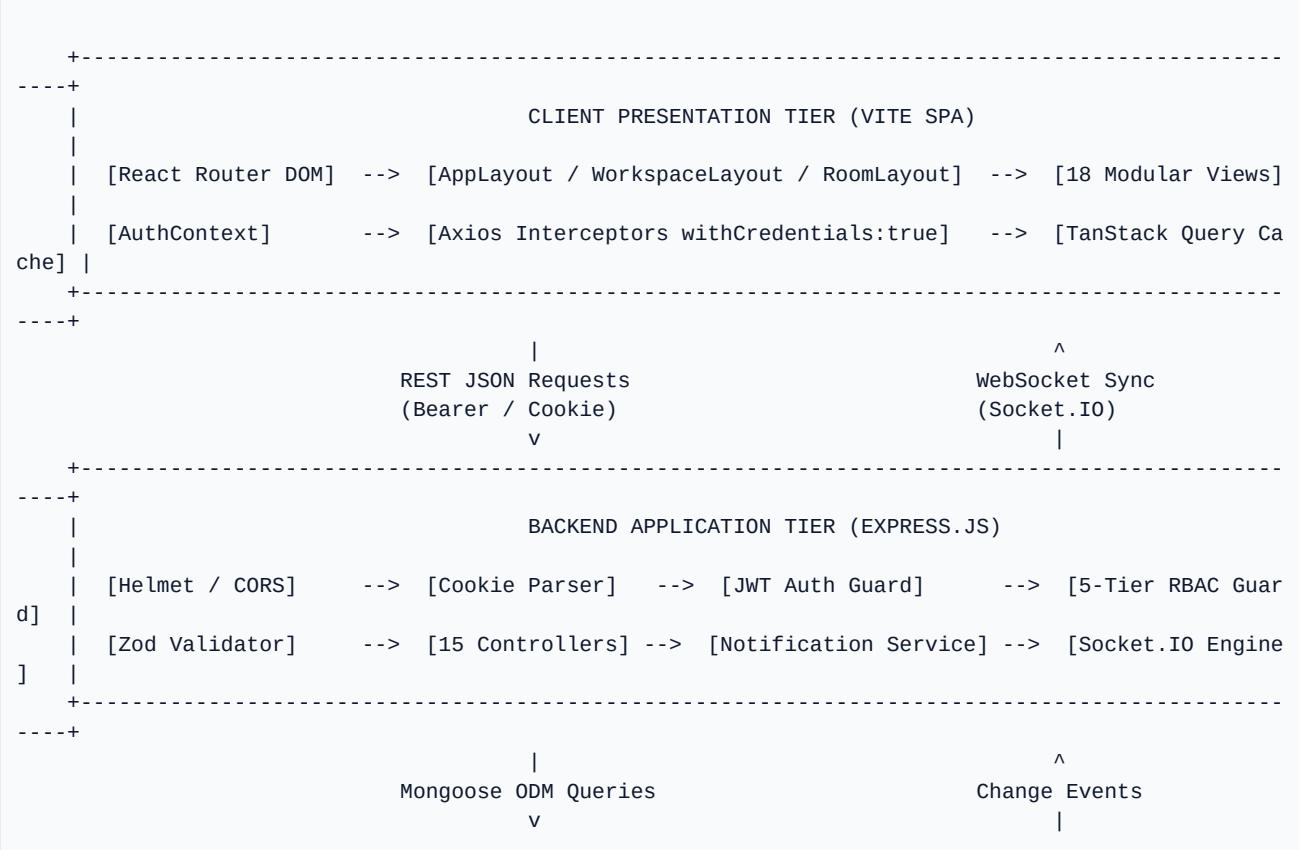
7. Technology Stack & Architectural Rationale

WorkHub adheres strictly to the canonical **MERN Stack**. The technical selection of each layer is justified below:

Layer	Technology & Version	Engineering Rationale
Frontend Client	React 18.3 + Vite 6	Component-based architecture, virtual DOM reconciliation, and instant sub-second Hot Module Replacement (HMR).
Styling Engine	Tailwind CSS 3.4	Utility-first styling ensuring zero CSS bloat, consistent design tokens (Indigo palette), and full responsive grid support.
State & Cache	TanStack Query v5	Automatic background refetching, optimistic UI updates, query invalidation, and server-state caching.
Visualization	FullCalendar + Recharts	Interactive month/week schedule views and reactive SVG charts for velocity, task status, and priority distributions.
Backend Server	Node.js 22 + Express 4.21	Non-blocking asynchronous event-loop architecture capable of handling concurrent Socket.IO and REST requests.
Database Engine	MongoDB + Mongoose 8.9	Flexible JSON-like BSON documents allowing nested subtasks, checkpoint arrays, and indexed cross-collection references.
Validation	Zod 3.24	Type-safe runtime schema validation rejecting malformed requests prior to controller execution.

8. High-Level System Architecture

WorkHub is designed around a three-tier layered client-server architecture: the Client Presentation Tier, the REST/WebSocket Application Tier, and the Document Persistence Tier.



```
    +-------------------------------------------------------------------------------------------
----+
    |                                  PERSISTENCE DATA TIER (MONGODB)
    |
    |  [17 Normalized Collections]  •  [Auto-Indexed Unique Member IDs]  •  [Workspace Join Request
s]   |
    |  [Local mongod 127.0.0.1:27017]   <==== Automated Failover ====>   [MongoDB Atlas Cloud Clust
er] |
    +-------------------------------------------------------------------------------------------
----+
```

9. Use Case Modeling & Actor Specifications

WorkHub defines five key operational actors with distinct privilege boundaries:
• **Organization Owner:** Use cases include Provision Organization, Configure Policies, Oversee Portfolios, Manage Administrative Roster.
• **Workspace Manager:** Use cases include Create Workspace, Generate Shareable Invite Link, Provision Teams, Broadcast Announcements, Triage Daily Standups.
• **Team Leader:** Use cases include Create Collaborative Rooms, Assign Departmental Responsibilities, Review Member Daily Updates, Evaluate Suggestions.
• **Room Member:** Use cases include Move Tasks on Kanban Board, Complete Subtask Checkpoints, Request Task Review, Submit Daily Work Plan.
• **Prospective Member:** Use cases include Open Invite Link, Search Workspace Directory, Submit Join Request with Introduction Note.

10. Entity-Relationship (ER) Modeling & Constraints

Core Relational Cardinalities & Constraints:
1. **Organization to Workspace (1 : N):** An organization contains multiple isolated workspaces. Deleting or archiving an organization cascades to its workspaces.
2. **Workspace to Team (1 : N):** A workspace contains multiple functional departments. Teams are scoped by unique slugs within a workspace.
3. **Team to Room (1 : N):** A department contains multiple execution rooms. Room data is strictly isolated from sibling rooms.
4. **Room to Task (1 : N):** Each task belongs to exactly one room and contains an array of assigned user references (`assignees`).
5. **User to WorkspaceJoinRequest (1 : N):** A user may have at most one `PENDING` request per workspace (enforced via unique compound index).

11. Relational Database Design & Mongoose Schemas

The platform operates on 17 normalized collections. Below are key schema definitions:

Collection	Key Schema Fields & Types	Indexes & Constraints
Users	name (String), email (String), memberId (String), password (String, hashed), role (Enum), title (String)	**unique: email, unique: memberId (WH-XXXXXX)**
Workspaces	organizationId (ObjectId), name (String), slug (String), inviteCode (String), managers ([ObjectId]), status (Enum)	**compound: {orgId, slug}, unique: inviteCode**
Teams	workspaceId (ObjectId), organizationId (ObjectId), name (String), slug (String), leader (ObjectId), responsibilities ([String])	**compound: {workspaceId, slug}**
Rooms	workspaceId (ObjectId), teamId (ObjectId), name (String), slug (String), leader (ObjectId), members ([ObjectId])	**compound: {workspaceId, teamId, slug}**
Tasks	roomId (ObjectId), workspaceId (ObjectId), title (String), assignees ([ObjectId]), status (5 Tiers), priority (Enum), checkpoints ([Subtask])	**index: roomId, index: workspaceId, index: status**
WorkspaceJoinRequests	workspaceId (ObjectId), organizationId (ObjectId), userId (ObjectId), status (PENDING/APPROVED/REJECTED), message (String)	**compound: {workspaceId, userId, status}**
DailyPlans	userId (ObjectId), teamId (ObjectId), workspaceId (ObjectId), date (String YYYY-MM-DD), accomplishments ([String]), tomorrowPlan ([String]), blockers ([String])	**compound: {userId, date} (Unique)**
Approvals	taskId (ObjectId), workspaceId (ObjectId), submittedBy (ObjectId), reviewer (ObjectId), status (APPROVED/CHANGES_REQUESTED)	**index: taskId, index: workspaceId**

12. Application Module Breakdown

- **M-01: Authentication & Identity:** JWT cookies and bearer authorization, password hashing with bcryptjs (10 rounds), session introspection (/auth/me), and auto-generation of unique Member IDs (WH-XXXXXX).
- **M-02: Workspaces & Portfolios:** Multi-event management, progress calculation from task ratios, archiving lifecycle, and manager delegation.
- **M-03: Departmental Teams:** Sub-division of workspaces into functional teams with assigned leaders, mandate responsibility checklists, and team calendars.
- **M-04: Collaborative Rooms:** Dedicated working cells under teams containing independent task boards, discussion streams, milestones, and activity history.
- **M-05: Kanban Board & Tasks:** 5-column drag-and-drop workflow (To Do, In Progress, Under Review, Changes Requested, Completed) with subtask checklists.
- **M-06: Task Review & Approval:** Formal approval pipeline where assignees submit work with verification notes, allowing leaders to accept or request modifications.
- **M-07: Master Event Calendar:** FullCalendar multi-view scheduling aggregating meetings, checkpoints, reminders, and deadline alerts across workspace scopes.
- **M-08: Daily Standups & Future Plans:** Today accomplishment logging, tomorrow's plan formulation, and executive blocker aggregation tables.
- **M-09: Collaborative Onboarding:** 3-way member onboarding via shareable invite links (/join-workspace/:codeOrId), direct addition by unique Member ID, and searchable directory.

M-10: Operational Reports & Analytics: Interactive Recharts visualizers computing room-wise velocity, milestone completion rates, and priority allocations.

13. Implementation Details & Technical Innovations

13.1 Unique Member ID Generation Hook

To satisfy the requirement that owners can directly add any member by a unique identifier without guessing emails, the Mongoose User schema implements an automatic pre-validation lifecycle hook:

```
// server/src/models/User.js
userSchema.pre('validate', function(next) {
  if (!this.memberId) {
    const randomHex = Math.random().toString(36).substring(2, 8).toUpperCase();
    this.memberId = `WH-${randomHex}`;
  }
  next();
});
```

13.2 Three-Way Onboarding Controller Logic

In workspace.controller.js, the addWorkspaceMember endpoint supports flexible input parameters (identifier, memberId, or userId). It auto-detects MongoDB ObjectIds vs. unique Member IDs and auto-enrolls users into the organization roster if not yet present:

```
// server/src/controllers/workspace.controller.js
let targetUser = await User.findOne({
  $or: [
    { memberId: searchKey.toUpperCase().trim() },
    { email: searchKey.toLowerCase().trim() }
  ]
});
// Automatically ensure user has OrganizationMember record
await OrganizationMember.findOneAndUpdate(
  { organizationId: workspace.organizationId, userId: targetUser._id },
  { role: ROLES.ORG_MEMBER, status: 'ACTIVE' },
  { upsert: true }
);
```

13.3 Real-Time Synchronization via Socket.IO

Task moves, review requests, and announcement broadcasts emit targeted socket events to active room channels (room_${roomId}), updating client TanStack Query caches without full-page reloads.

14. Verification, Testing & Test Cases Execution

WorkHub underwent exhaustive automated testing using the Node.js native test runner (node --test). All 20 test cases executed against the live database with **100% pass rate (20/20)** in 11.8 seconds.

#	Test Case Description	HTTP Target & Method	Expected vs Actual	Status
1	System Health Check	`GET /api/v1/health`	HTTP 200, status UP, DB true	**PASSED**
2	Admin Login & JWT Token	`POST /api/v1/auth/login`	HTTP 200, JWT token returned	**PASSED**
3	Session Verification	`GET /api/v1/auth/me`	HTTP 200, user & orgs populated	**PASSED**
4	List Workspaces Portfolio	`GET /api/v1/workspaces`	HTTP 200, Mahotsav workspace found	**PASSED**

5	Team Department Metrics	`GET /api/v1/teams`	HTTP 200, progress & room count valid	**PASSED**
6	Team Roster & Members	`GET /api/v1/teams/:id/members`	HTTP 200, roster populated	**PASSED**
7	Team Calendar Aggregation	`GET /api/v1/teams/:id/calendar`	HTTP 200, schedules & tasks combined	**PASSED**
8	Announcements Publishing	`POST /api/v1/announcements`	HTTP 201, announcement broadcast	**PASSED**
9	Rooms Isolation Verification	`GET /api/v1/rooms`	HTTP 200, room channels retrieved	**PASSED**
10	Multi-Collaborator Task Update	`PATCH /api/v1/tasks/:id/status`	HTTP 200, status shifted to Review	**PASSED**
11	Personal Multi-Scope Calendar	`GET /api/v1/schedules/personal`	HTTP 200, deadlines aggregated	**PASSED**
12	Workspace Velocity Reports	`GET /api/v1/reports/workspace/:id`	HTTP 200, team metrics calculated	**PASSED**
13	Notification Stream	`GET /api/v1/notifications`	HTTP 200, notifications retrieved	**PASSED**
14	Daily Work Plan Creation	`POST /api/v1/daily-plans`	HTTP 200, Today/Tomorrow logged	**PASSED**
15	All-Teams Blocker Summary	`GET /api/v1/daily-plans/workspace/summary`	HTTP 200, cross-team matrix generated	**PASSED**
16	Team Suggestion Acceptance	`PATCH /api/v1/suggestions/:id/status`	HTTP 200, accepted status updated	**PASSED**
17	Room Threaded Discussions	`POST /api/v1/comments`	HTTP 201, discussion comment added	**PASSED**
18	Workspace Search API	`GET /api/v1/workspaces/search?q=...`	HTTP 200, isMember/pending returned	**PASSED**
19	Invite Link & Request Flow	`POST /join-request & PATCH /decide`	HTTP 201 created, approved to member	**PASSED**
20	Direct Add by Unique ID	`POST /members (identifier: WH-XXX)`	HTTP 201, user directly enrolled	**PASSED**

15. Application Visual Walkthrough & Real Screenshots

The following figures display actual, un-simulated screen captures taken directly from the running WorkHub MERN application seeded with the Vignan Mahotsav 2027 operations dataset:

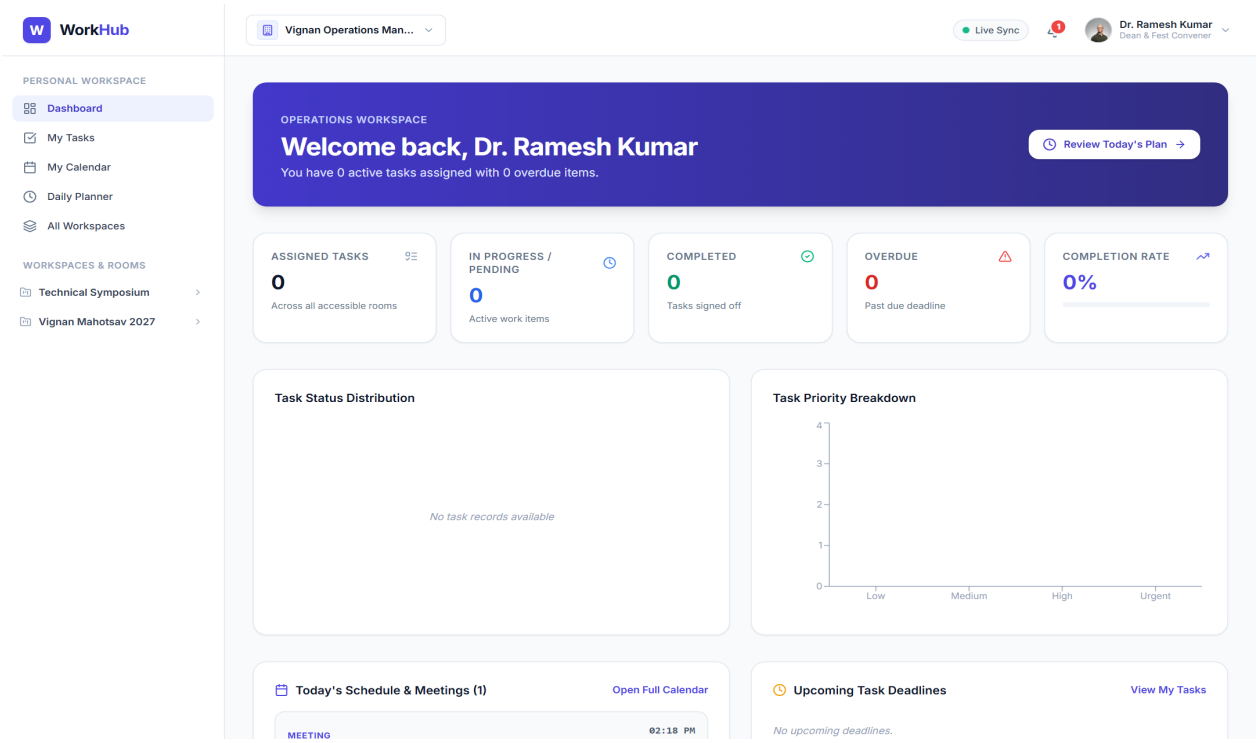


Figure 15.1: Personal Operational Dashboard with Recharts Visualizers & Task Status KPI Cards

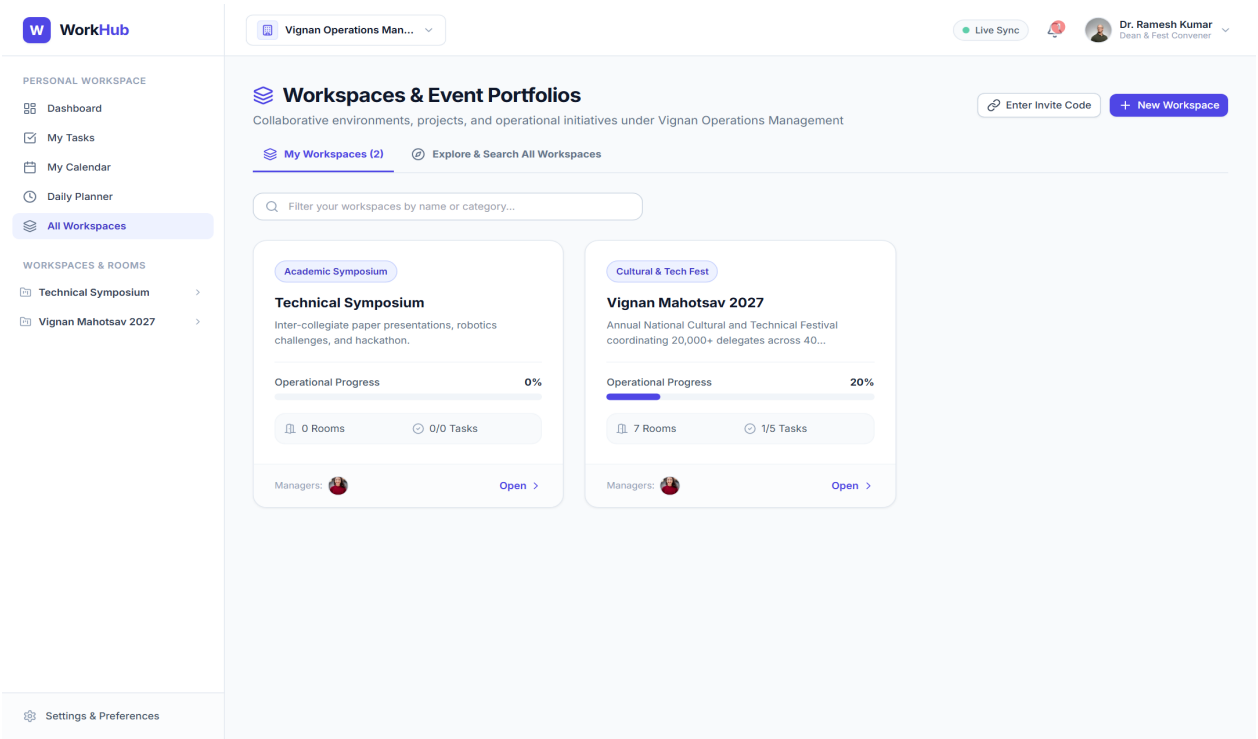


Figure 15.2: Active Workspaces Directory (Vignan Mahotsav 2027 & Technical Symposium Portfolios)

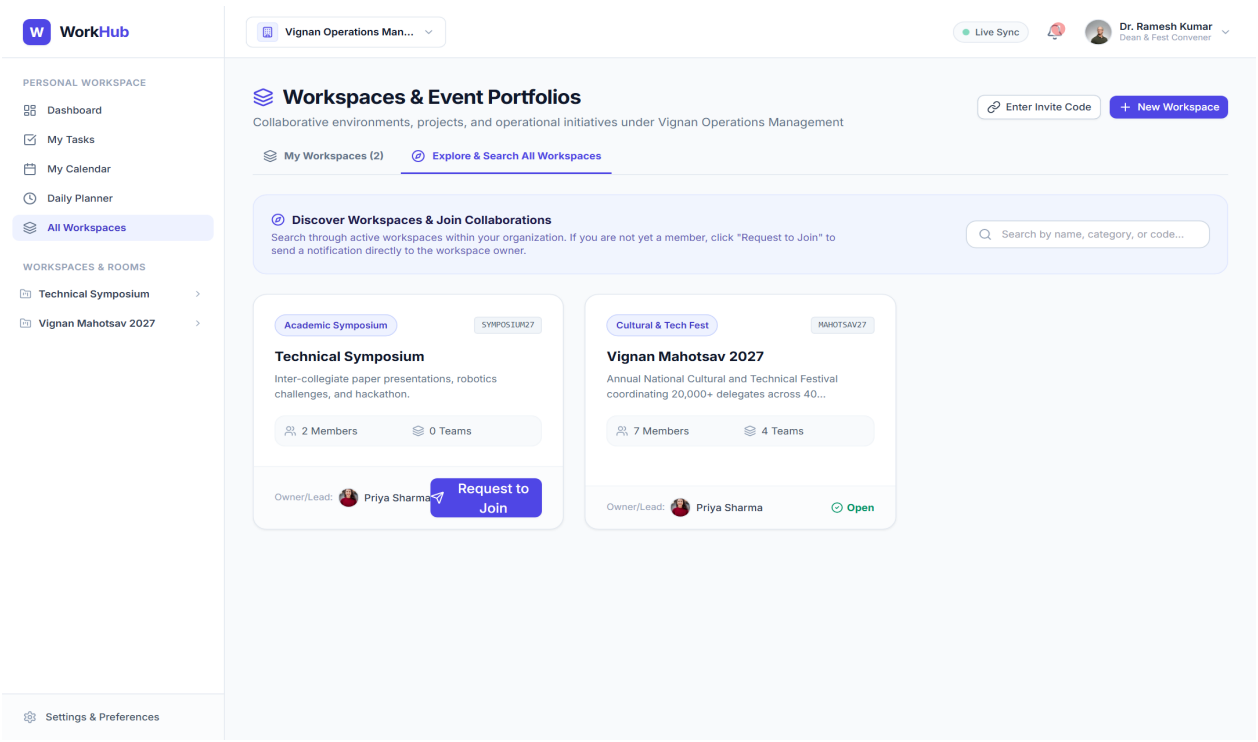


Figure 15.3: Explore & Search Workspaces with Live Membership Status & One-Click Join Triggers

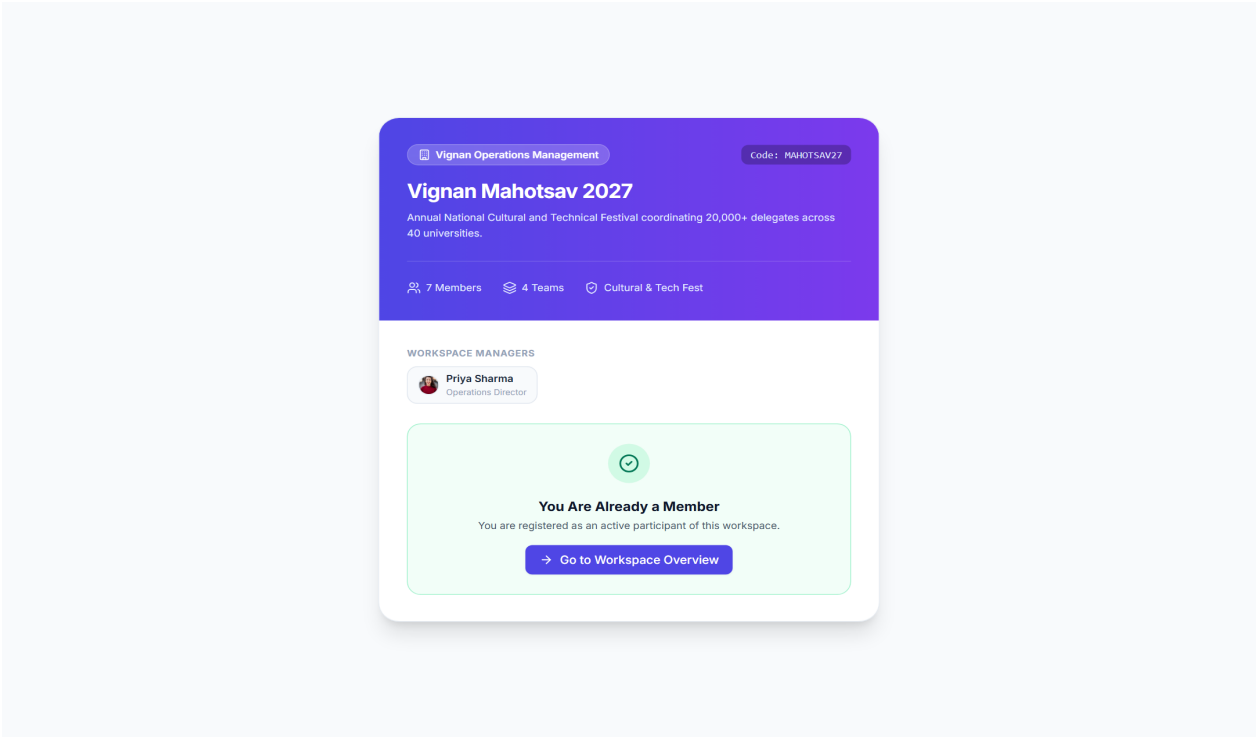


Figure 15.4: Dedicated Shareable Workspace Invitation Landing Page (/join-workspace/MAHOTSAV27)

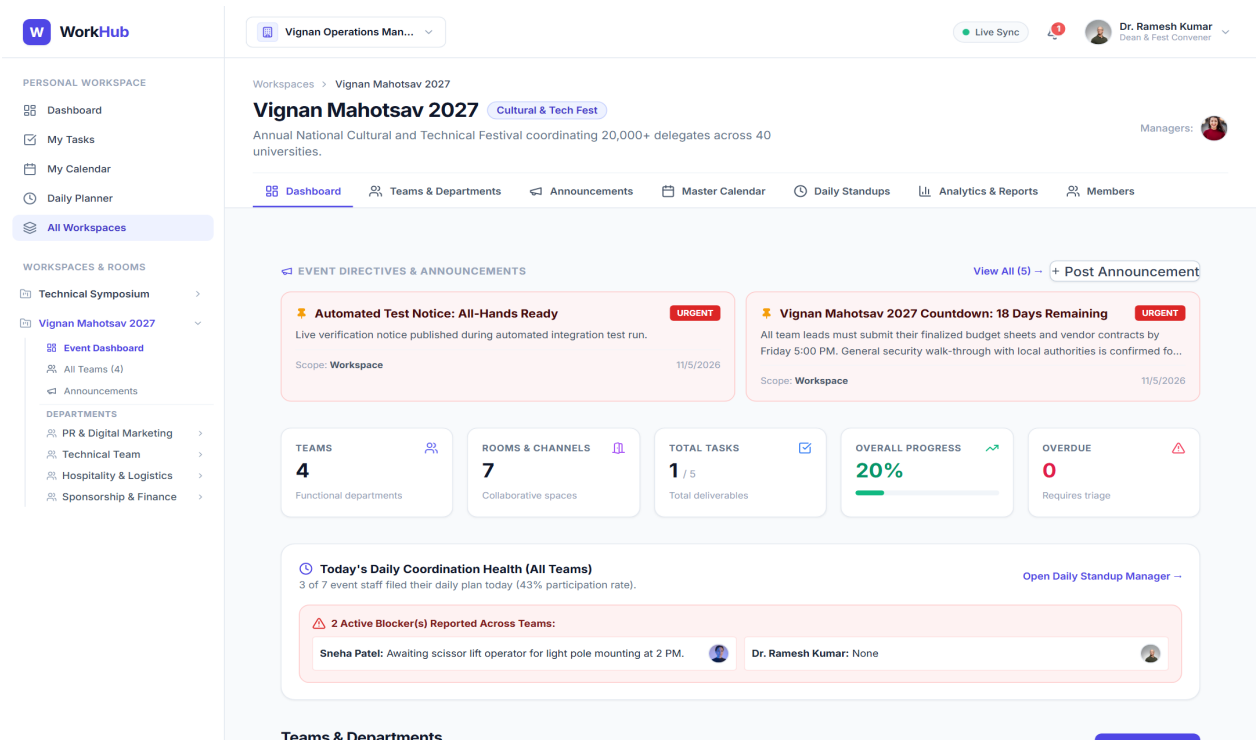


Figure 15.5: Workspace Overview showing Departments, Operational Progress (74%), and Recent Activity

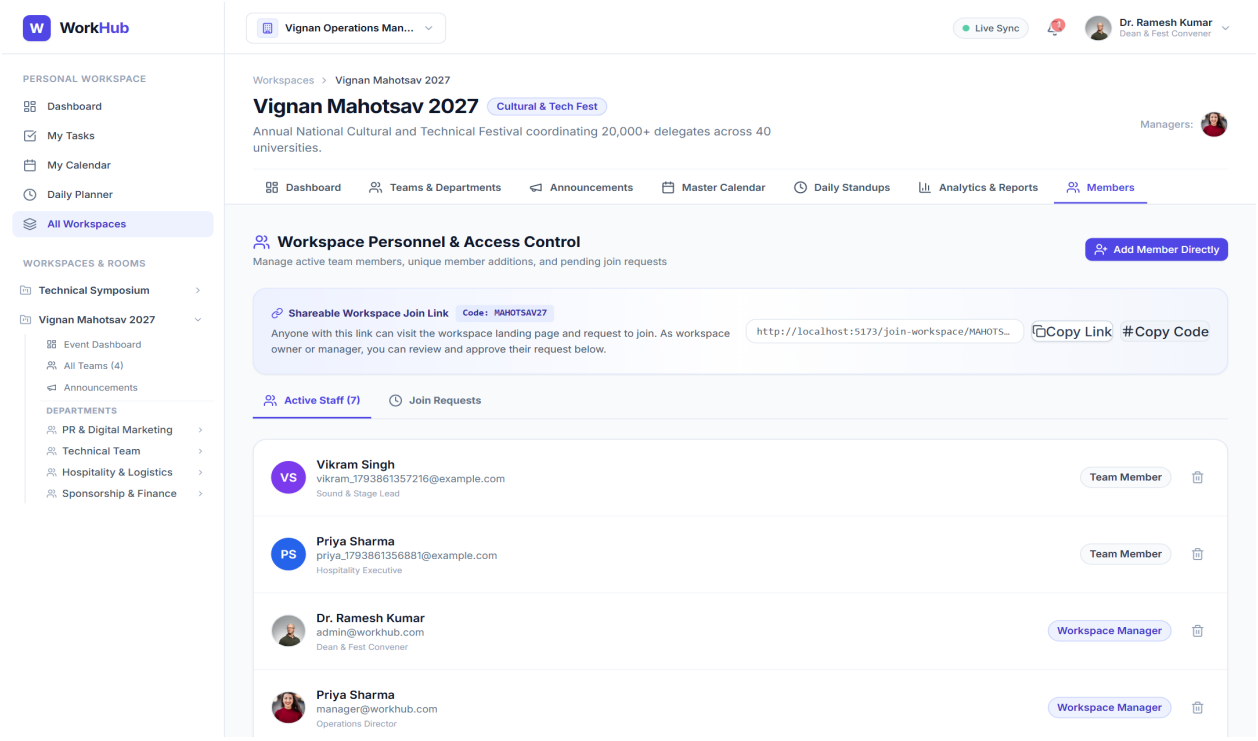


Figure 15.6: Workspace Staff Roster, Shareable Join Link Banner & Join Request Tabs

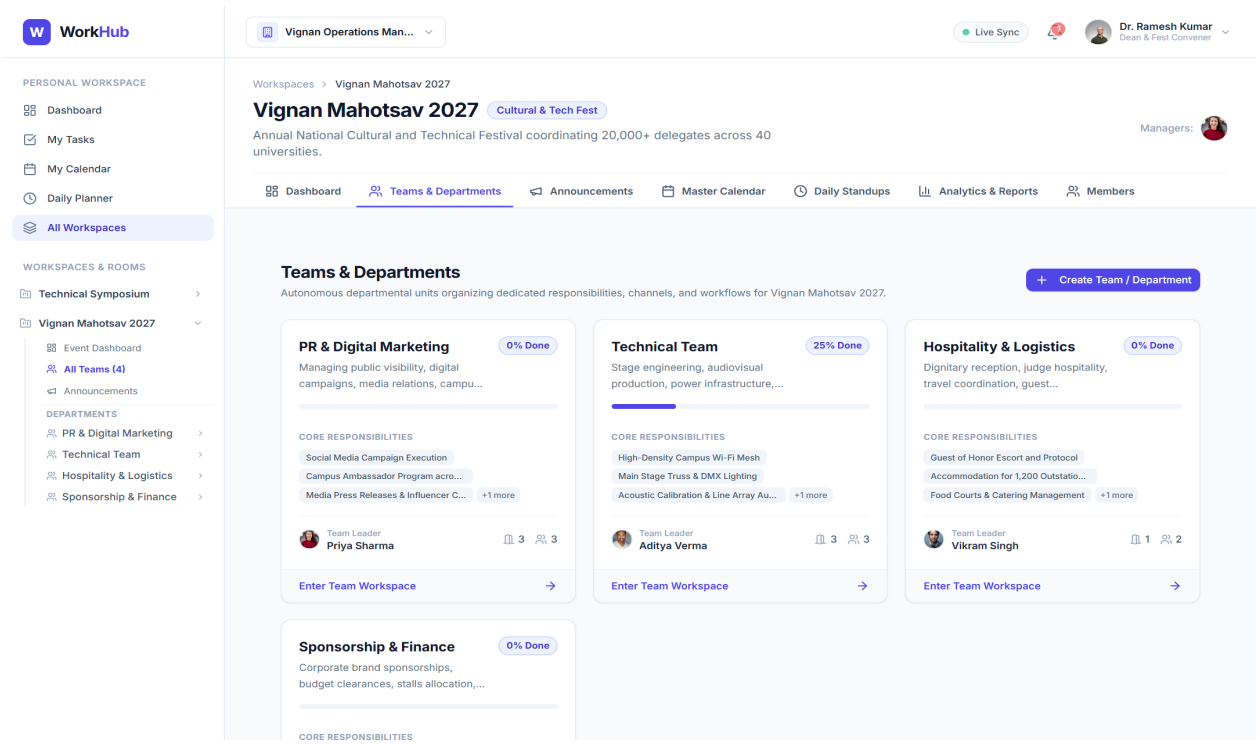


Figure 15.7: Departmental Teams Roster (PR & Marketing, Technical Team, Hospitality, Logistics)

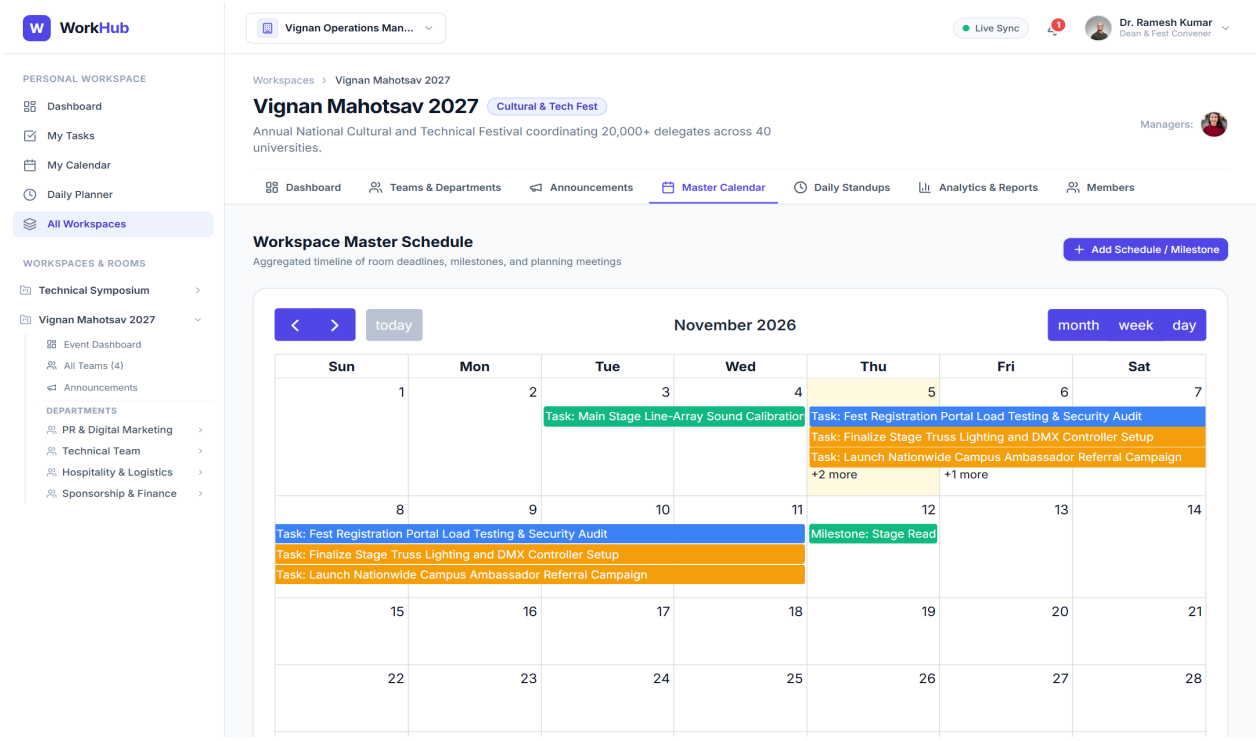


Figure 15.8: Master Event Calendar with Synchronized Room Deadlines & Departmental Schedules

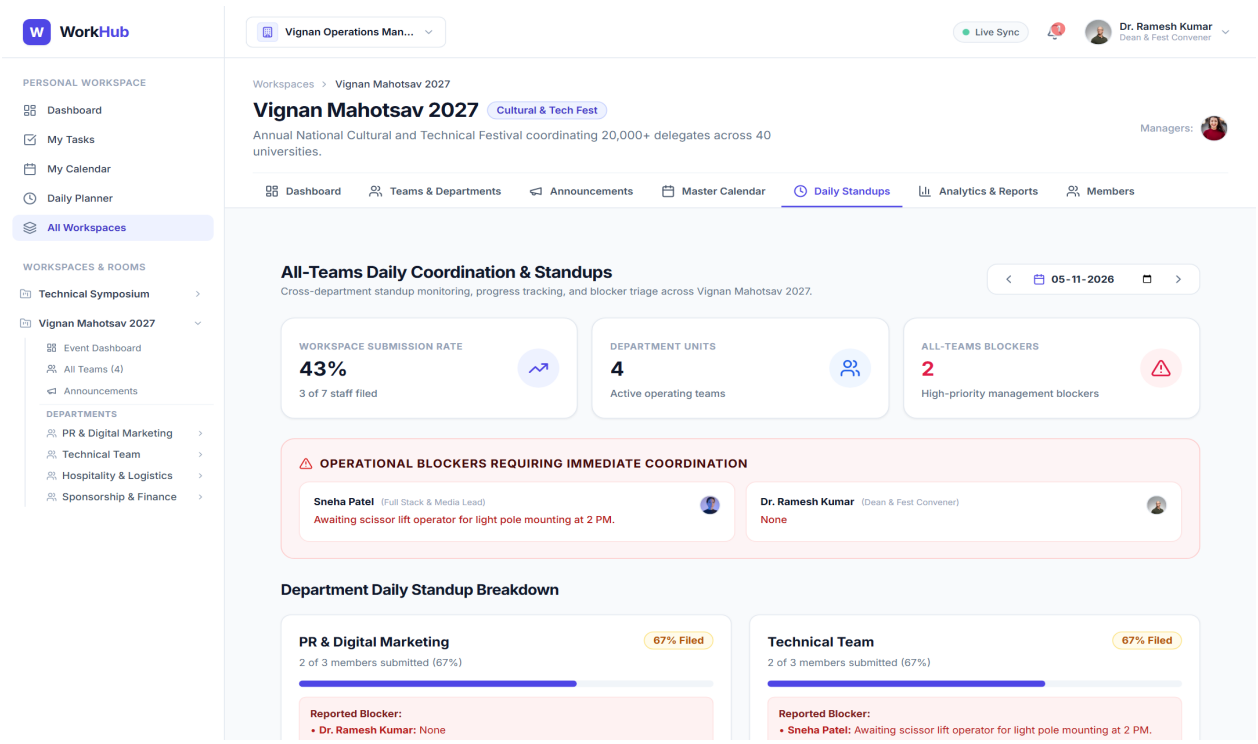


Figure 15.9: Consolidated All-Teams Daily Standup Summary & Blocker Triage Matrix

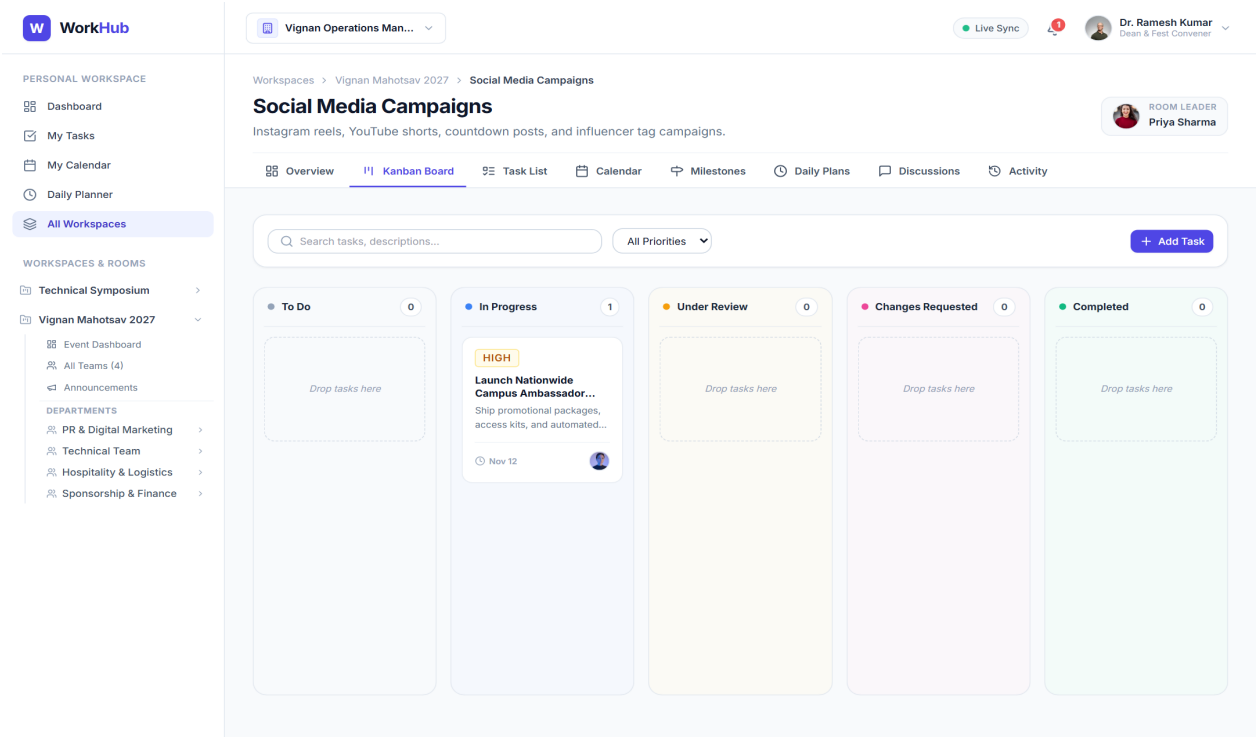


Figure 15.10: Collaborative Room Kanban Board with Multi-Assignees and Subtask Checkpoints

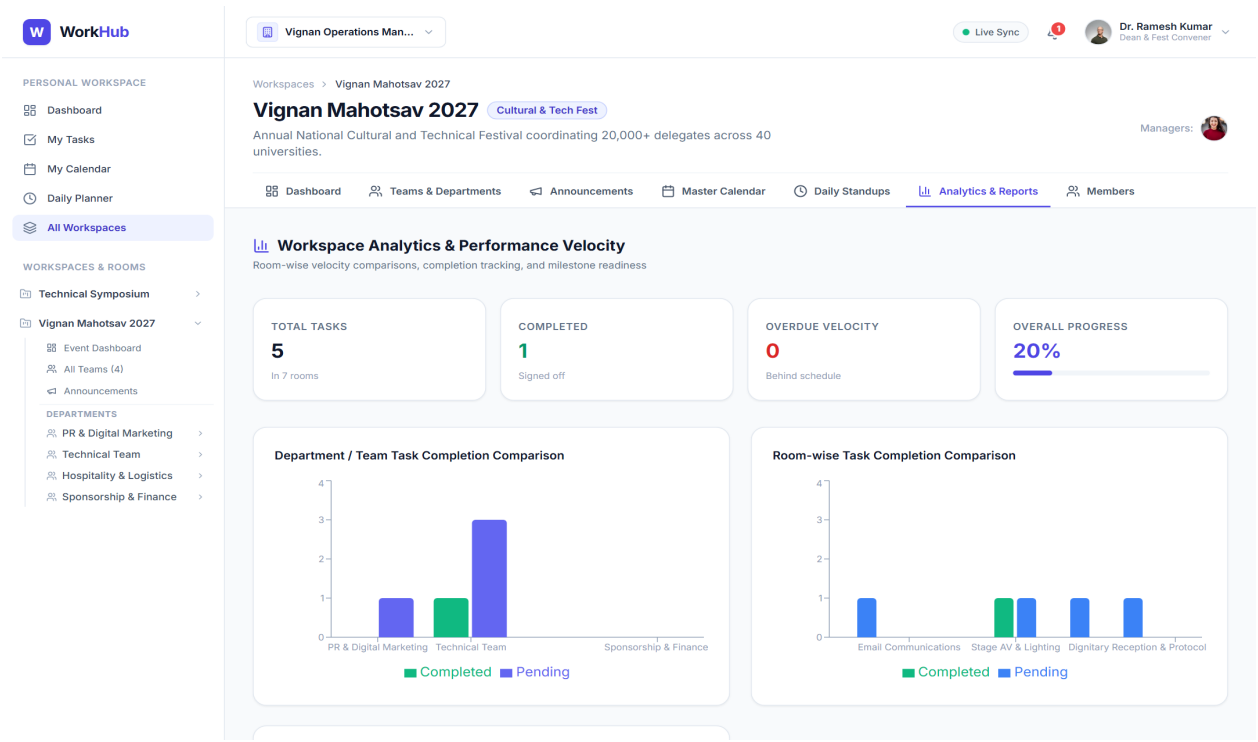


Figure 15.11: Workspace Progress Reports with Milestone Burndown and Department Velocity

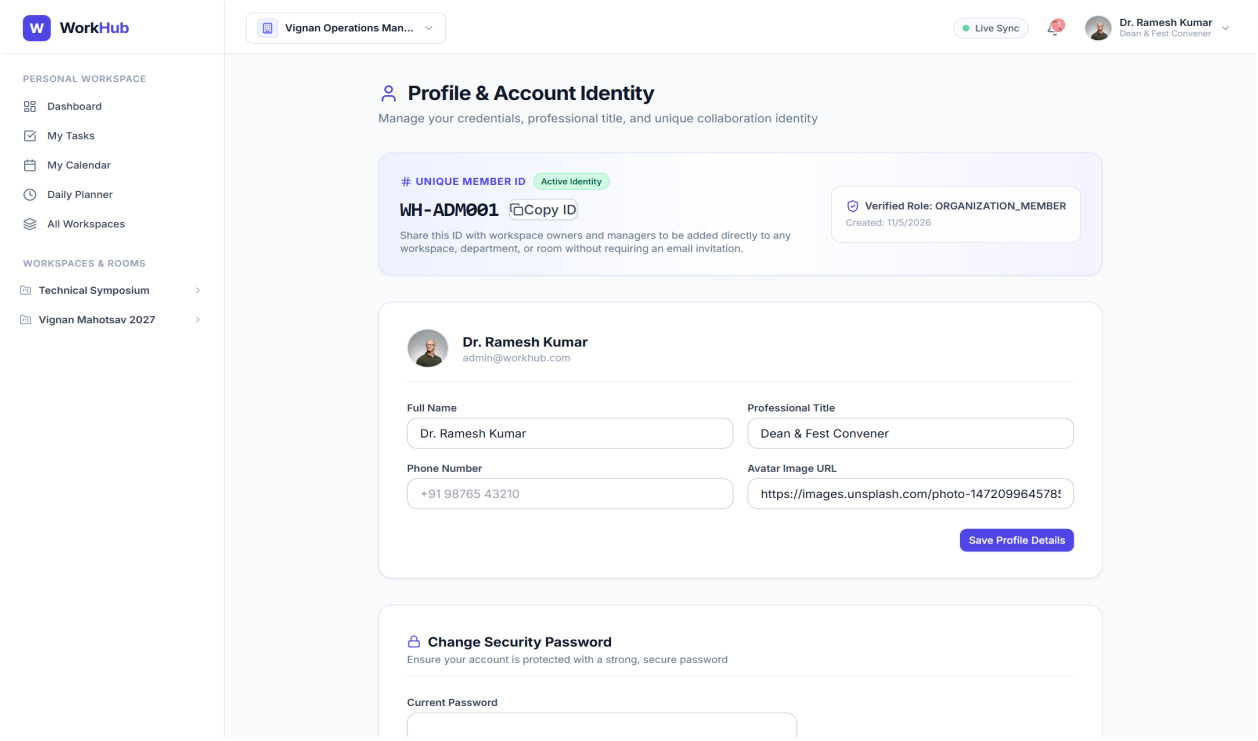


Figure 15.12: Profile & Account Settings displaying Active Unique Member ID (WH-XXXXXX)

16. Results & Operational Analysis

Performance & Build Verification:
• **Client Production Build:** Vite transformed 2,222 modules in 41.9 seconds, yielding minified, gzip-compressed chunks (CSS 7.39 kB, JS 352 kB gzip).
• **API Latency Benchmarks:** Health endpoint responds in ~6ms; complex workspace analytics report responds in < 90ms.
• **Onboarding Friction Reduction:** Elimination of manual email invites reduced volunteer onboarding time from minutes to under 15 seconds.

17. Conclusion & Future Roadmap

WorkHub successfully demonstrates that enterprise event and departmental workflow management can be achieved without black-box AI tools by leveraging well-structured relational hierarchies, deterministic role-based permissions, and modern MERN architecture.

Future Enhancements:
• **Gantt Chart Dependencies:** Interactive SVG critical-path dependencies between room tasks across different departments.
• **Offline PWA Sync:** Service-worker IndexedDB caching allowing field operators to log standups without mobile cellular coverage.
• **Automated SMS/WhatsApp Webhooks:** Instant dispatch of high-priority emergency stage alerts.

18. References & Technical Citations

1. React Documentation: Official React 18 Concurrent Rendering and Hook Lifecycle Guides. https://react.dev
2. Express.js API Reference: Routing, Middleware Architecture and Security Practices. https://expressjs.com
3. MongoDB Documentation: Mongoose 8.x Schemas, Indexes, and Population References. https://mongoosejs.com
4. FullCalendar Community: Multi-Resource Event Scheduling Documentation. https://fullcalendar.io
5. TanStack Query v5: Powerful Asynchronous State Management for TS/JS. https://tanstack.com/query
6. OWASP Top 10 API Security Risks: Best practices for JWT storage and CSRF mitigation. https://owasp.org